

VNUHCM - UNIVERSITY OF SCIENCE

FACULTY OF INFORMATION TECHNOLOGY

CSC14003 – STATISTICAL MACHINE LEARNING



FINAL PROJECT - Xây dựng hệ thống hỏi–đáp trích xuất tiếng Việt

LỚP 23CNTThuc2

Sinh viên:

23127168 – Cao Trần Bá Đạt
23127120 – Trần Danh Thiện

Giảng viên:

Mr. Ngô Minh Nhựt
Mr. Lê Long Quốc

August 20th, 2026

Contents

1	Giới thiệu và bài toán	3
2	Dataset và phân tích dữ liệu	3
2.1	Nguồn dữ liệu và cấu trúc một bản ghi	3
2.2	Làm sạch, kiểm tra toàn vẹn và chia tách	4
2.3	Thống kê mô tả: độ dài và hệ quả thiết kế	5
3	Mô hình nền và quá trình huấn luyện	6
4	Kỹ thuật đề xuất: cấu trúc một tầng và cấu trúc hai tầng	7
4.1	Cấu trúc một tầng	7
4.2	Cấu trúc hai tầng	8
4.3	Bảng so sánh hai cấu trúc	11
5	Kết quả thực nghiệm và thảo luận	11
5.1	Thiết lập đánh giá	11
5.2	Chất lượng của mô hình nền trên test ($n = 2882$)	12
5.3	So sánh một tầng và hai tầng bằng số liệu	13
5.4	Giới hạn	14
6	Ứng dụng web	15
6.1	Kiến trúc: một lõi suy luận, hai giao diện	15
6.2	Triển khai và giới hạn của bản demo	17
7	Kết luận	17
8	References	18
	Tài liệu tham khảo	18

List of Tables

1	Kích thước và phân bố ba tập sau khi làm phẳng và làm sạch	4
2	Phân bố độ dài tính theo số từ (toàn bộ 28.456 câu; đáp án tính trên 19.239 câu có đáp án)	5
3	Độ dài theo token <code>PhobertTokenizer</code> (ngữ cảnh: 4.101 đoạn duy nhất; đáp án: 19.239 câu)	5
4	Cấu hình mô hình và huấn luyện	7
5	Diễn biến <code>eval_loss</code> theo epoch trên validation	7
6	Ba mươi tư đặc trưng bề mặt của một ứng viên, theo sáu nhóm; cột Chiều là số thành phần đóng góp vào véc tơ 34 chiều	9
7	Đôi chiếu thiết kế giữa cấu trúc một tầng và cấu trúc hai tầng	11
8	Môi trường thực nghiệm	12
9	Kết quả mô hình một tầng trên UIT-ViQuAD 2.0 test	12
10	Hồ sơ lỗi của mô hình một tầng trên test ($n = 2882$, các nhóm loại trừ nhau)	13
11	Một tầng (pointer thuần) và hai tầng (reranker + ngưỡng $\tau = 0.95$) — test, $n = 200$	13
12	Quét ngưỡng τ trên validation ($n = 2872$)	14
13	Hai endpoint của <code>src/api.py</code> (đã kiểm chứng bằng <code>curl</code>)	17

1 Giới thiệu và bài toán

Tìm kiếm từ khoá trên văn bản tiếng Việt trả về cả tài liệu dài và để người dùng tự đọc. Bài toán *Hỏi-Đáp trích xuất* (Extractive QA) yêu cầu máy đọc một đoạn ngữ cảnh và trả về đúng một chuỗi con của đoạn đó chứa câu trả lời. Đề án xây một hệ thống như vậy cho tiếng Việt, huấn luyện và chạy giao diện trên một GPU cá nhân 4GB.

Bài toán phát biểu theo chuẩn SQuAD 2.0 [3] với dữ liệu UIT-ViQuAD 2.0 [1, 2]. Cho ngữ cảnh $\mathbf{C} = (c_1, \dots, c_m)$ và câu hỏi $\mathbf{Q} = (q_1, \dots, q_k)$, hệ thống phải trả về một trong hai kết quả:

- một span liên tục $\mathbf{A} \subseteq \mathbf{C}$ xác định bởi cặp chỉ số (start, end), nếu câu hỏi trả lời được (`is_impossible = False`);
- kết quả rỗng $\mathbf{A} = \emptyset$, nếu ngữ cảnh không chứa thông tin để trả lời (`is_impossible = True`).

Hai ràng buộc, đều đo trên chính bộ dữ liệu (mục 2): 31.0% câu có ngữ cảnh không nhét vừa một cửa sổ 256 token, và 32.4% câu là câu bẫy, chỉ khác một câu hỏi thật ở một chi tiết then chốt đã bị thay. Hệ thống chỉ dựa vào trùng lặp từ khoá sẽ sai ở đúng một phần ba số câu, nên *buộc* phải biết nói “không có đáp án”. Đề án coi đây, chứ không phải độ chính xác khi định vị, là chỗ yếu nhất của mô hình nền.

Bốn đóng góp của đề án:

- Pipeline tự viết phần ánh xạ toạ độ: làm phẳng JSON sang Parquet, tự trải cửa sổ trượt, tự sinh `offset_mapping` ký tự-token cho PhoBERT thay vì dùng `return_offsets_mapping` của thư viện.
- Phân tích dữ liệu đo tới từng bản ghi, gồm phát hiện và sửa 153 vị trí `answer_start` lệch trong JSON gốc.
- Hồ sơ lỗi định lượng phân hoạch đủ 2.882 câu test thành tám nhóm loại trừ nhau.
- **Kỹ thuật đề xuất:** tăng hai chấm lại pool ứng viên của extractor kèm *ngưỡng từ chối có hiệu chuẩn* τ (mục 4.2), tăng +21.21 điểm bắt câu bẫy so với cấu trúc một tầng trên mẫu 200 câu test (mục 5.3).

2 Dataset và phân tích dữ liệu

2.1 Nguồn dữ liệu và cấu trúc một bản ghi

UIT-ViQuAD 2.0 ghép từ hai nguồn: câu hỏi có đáp án do nhóm UIT xây dựng và duyệt thủ công [1], câu hỏi *bẫy* lấy từ thách thức ViMRC của VLSP 2021 [2]. Cả hai theo khung SQuAD 2.0 [3].

Dữ liệu gốc là cây JSON ba tầng `title` → `paragraphs` → `qas`: 138 bài viết, 4.101 đoạn `context`, mỗi câu hỏi mang `id`, `question`, `is_impossible` và `answers`. Mỗi đáp án có `text` và `answer_start` là vị trí ký tự trong `context`; câu bẫy có thêm `plausible_answers` — phương án *nghe* hợp lý nhưng sai ở chi tiết đã bị thay.

Ba tính chất của cây JSON quyết định mọi bước sau:

- **Mỗi câu hỏi chỉ có một đáp án** (0 câu có nhiều hơn một `answers`), nên quy tắc “lấy điểm cao nhất trên mọi đáp án vàng” của SQuAD suy biến thành chấm một đáp án duy nhất.
- **Một đoạn văn gánh nhiều câu hỏi**: trung bình 6.9 câu/đoạn, nhiều nhất 11 — lý do phải chia `train/test` theo *nhóm ngữ cảnh* (mục 2.2).
- **Tọa độ đáp án là ký tự, không phải token**, nên lỗi định vị chỉ phát hiện được bằng cách cắt chuỗi và so sánh.

2.2 Làm sạch, kiểm tra toàn vẹn và chia tách

Lỗi lệch tọa độ trong dữ liệu gốc. Cắt `context[answer_start : answer_start + len(text)]` trên 19.240 đáp án của JSON gốc cho 19.086 trường hợp (99.20%) khớp chính xác và 154 trường hợp (0.80%) lệch. Bộ làm sạch `ViQuADCleaner` trong `src/data_preprocessing` tìm lại chuỗi đáp án trong cửa sổ ± 20 ký tự quanh vị trí khai báo: 153 câu định vị lại thành công, 1 câu không tìm thấy nên bị loại. JSON gốc có 28.457 câu hỏi; sau bước này còn 28.456, và mọi số liệu trong báo cáo tính trên số đó. Kiểm chứng lại: 19.239/19.239 đáp án có `answer_start` dừng lại đúng `answer_text`, tức không còn nhãn vị trí sai nào lọt vào huấn luyện.

Các kiểm tra toàn vẹn còn lại. Không có giá trị null ở trường bắt buộc; không có `context` hay `question` rỗng; 0 trùng lặp mã `qa_id`; 3 trùng lặp cặp nội dung (`context` + `question`), phân bố 2 ở `train`, 1 ở `val`, 0 ở `test`.

Chia tách. Dùng `GroupShuffleSplit` của `scikit-learn` [11] theo nhóm `context_id` với `random_state = 42`, tỉ lệ 80/10/10; `context_id = {title}_{chỉ số đoạn}`. Chia theo nhóm ngữ cảnh là bắt buộc: chia theo câu sẽ để cùng một đoạn vào cả `train` lẫn `test`, và điểm `test` khi đó đo học vẹt. Đo thực tế xác nhận 0 **đoạn ngữ cảnh chung giữa `train` và `test`**. Ngược lại, 110/138 *chủ đề* (`title`) xuất hiện ở cả hai tập, tức `test` không rời khỏi miền chủ đề của `train` (mục 5.4).

Table 1: Kích thước và phân bố ba tập sau khi làm phẳng và làm sạch

Tập	Câu hỏi	Có đáp án	Câu bẫy	% bẫy	Ngữ cảnh	Câu/đoạn
Train	22.702	15.366	7.336	32.31	3.279	6.92
Val	2.872	1.935	937	32.63	411	6.99
Test	2.882	1.938	944	32.76	411	7.01
Tổng	28.456	19.239	9.217	32.39	4.101	6.94

Tỉ lệ câu bấy gần như không đổi giữa ba tập (32.3–32.8%), nên không cần trọng số hoá khi huấn luyện và mọi con số NoAns được đo trên cùng một điều kiện khó.

2.3 Thống kê mô tả: độ dài và hệ quả thiết kế

Độ dài đo bằng *số từ* (cách người dùng nhìn dữ liệu) và *số token PhoBERT* (trả lời câu hỏi duy nhất về kỹ thuật: đoạn có nhét vừa một cửa sổ model không).

Table 2: Phân bố độ dài tính theo số từ (toàn bộ 28.456 câu; đáp án tính trên 19.239 câu có đáp án)

Đại lượng	Trung bình	Trung vị	p90	p95	Min	Max
Ngữ cảnh	180.6	161	267	315	88	1.537
Câu hỏi	14.6	14	21	24	1	53
Đáp án	10.0	6	24	31	1	150

Table 3: Độ dài theo token `PhobertTokenizer` (ngữ cảnh: 4.101 đoạn duy nhất; đáp án: 19.239 câu)

Đại lượng	Trung bình	Trung vị	p90	p95	Max
Ngữ cảnh (token)	215.6	192	318	377	2.064
Đáp án (token)	11.7	7	27	37	221

Bốn kết luận ứng với bốn tham số của hệ thống:

- **Tiếng Việt phình token nhanh:** 180.6 từ tương ứng 215.6 token, tức ≈ 1.19 token/từ. Tính dung lượng phải tính bằng token: đoạn 220 từ đã vượt cửa sổ 256 token.
- **Cửa sổ trượt là bắt buộc:** 23.82% số đoạn một mình nó đã không vừa 252 token; tính cả câu hỏi thì 31.0% số câu, riêng test 32.03%, cần nhiều hơn một cửa sổ. Đây là cơ sở của `max_length = 256` và `doc_stride = 64`.
- **Đáp án rất ngắn nên biên độ sai rất đắt:** trung vị 6 từ (7 token), nên thừa hoặc thiếu *một từ* là mất EM và mất phần lớn F1. Đó là lý do nhóm lỗi cắt biên lớn (mục 5.2).
- **Trần độ dài đáp án gần như không cắt mất gì:** `max_answer_length = 64` chỉ loại 0.72% ứng viên (139/19.239).

Tiền xử lý. Cây JSON làm phẳng thành bảng hai chiều, mỗi dòng một câu hỏi (`qa_id`, `context_id`, `title`, `context`, `question`, `answer_text`, `answer_start`, `is_impossible`), rồi lưu sang `.parquet`: gọn và nạp nhanh hơn hẳn JSON khi lặp qua 28k mẫu, tương thích trực tiếp với `datasets` của Hugging Face [10].

3 Mô hình nền và quá trình huấn luyện

Encoder là **vinai/phobert-base**, một biến thể RoBERTa [5] tiền huấn luyện riêng cho tiếng Việt [4]. Khác biệt quan trọng nhất so với mô hình đa ngữ: PhoBERT tách *từ ghép* ngay từ giai đoạn tiền huấn luyện, nên một token thường là một đơn vị nghĩa thay vì một mảnh âm tiết tùy ý — đó cũng là lý do tỉ lệ token/từ chỉ ở mức 1.19. Vì bảng từ đã chứa sẵn đơn vị nghĩa, **đồ án không chạy công cụ tách từ ngoài** (như pyvi).

Trên tầng ẩn cuối $\mathbf{H} \in \mathbb{R}^{N \times 768}$, mô hình dùng head span kinh điển của BERT [6]: một chiếu tuyến tính $W \in \mathbb{R}^{2 \times 768}$ sinh hai véc tơ điểm số $\mathbf{z}^{\text{start}} = \mathbf{H}W_{\text{start}}^T$ và $\mathbf{z}^{\text{end}} = \mathbf{H}W_{\text{end}}^T$. Bài toán quy về hai lớp phân loại đa lớp song song, mất mát là trung bình hai cross-entropy:

$$\mathcal{L} = \frac{1}{2} \left(-\log P_{y^{\text{start}}}^{\text{start}} - \log P_{y^{\text{end}}}^{\text{end}} \right), \quad P_i^{\text{start}} = \frac{e^{z_i^{\text{start}}}}{\sum_j e^{z_j^{\text{start}}}}$$

Hai head được học *độc lập*. Điểm này quay lại ở mục 4 như lời giải thích cho nhóm lỗi mà mọi hậu xử lý đều không chữa được.

Chuỗi vào encoder có dạng `<s> Question </s></s> Context </s>`, do lớp `QATokenizerProcessor` (`src/train.py`) đảm nhiệm với ba quyết định thiết kế:

- **Cửa sổ trượt:** `max_length = 256`, `doc_stride = 64`. Câu hỏi mã hoá toàn bộ, ngữ cảnh còn ngân sách $256 - |\mathbf{Q}| - 4$ token. Mỗi bước trượt tiến `min(length, doc_stride)` token nên các cửa sổ đè phủ lên nhau, bảo đảm đáp án nằm ở biên vẫn nguyên vẹn trong ít nhất một cửa sổ. Chuỗi pad bằng `<pad>` kèm `attention_mask`.
- **Ánh xạ ký tự $\rightarrow$ token:** `PhobertTokenizer` của HuggingFace *không* hỗ trợ `return_offsets_mapping`, nên đồ án tự xây lại offset bằng cách duyệt tuần tự token, xoá ký tự nối `@@`, dò vị trí token trên văn bản gốc và cập nhật con trỏ. Kết quả là danh sách `(start_char, end_char)` cho từng token; token đặc biệt và phần câu hỏi được gán `(0, 0)`.
- **Gán nhãn:** `start_position` là token cuối cùng có điểm bắt đầu ký tự $\leq \text{answer_start}$, `end_position` là token đầu tiên có điểm kết thúc ký tự $\geq \text{answer_end}$. Nếu `is_impossible = True` hoặc cửa sổ không phủ *toàn bộ* đáp án, cả hai nhãn cùng trở về token `<s>` (index 0). Nhờ đó mô hình học được hành vi “im lặng” mà không cần head riêng, và mẫu bị cắt nửa không bị gán nhầm thành span sai.

Table 4: Cấu hình mô hình và huấn luyện

Hạng mục	Giá trị
Kiến trúc	12 tầng, 12 head, $d = 768$, <code>intermediate</code> 3072, tối đa 258 token
Bảng từ	64.001 ký hiệu (<code>PhobertTokenizer</code> : 64.000)
Tham số	134.409.218 (đã gồm hai head), 537.660.792 bytes FP32
Bộ tối ưu	AdamW [7], lr 3×10^{-5} , <code>warmup_ratio</code> 0.06, <code>weight_decay</code> 0.01
Lô	$8 \times$ <code>gradient_accumulation</code> 2 = 16 mẫu, 3 epoch = 6621 step
VRAM	FP16 + <code>gradient_checkpointing</code> [8] — điều kiện để chạy được trên 4GB
Chọn weights	<code>load_best_model_at_end</code> theo <code>eval_loss</code> , <code>seed</code> 42
Thời gian	3.537,8 giây ($\approx$ 59 phút)

Table 5: Diễn biến `eval_loss` theo epoch trên validation

Epoch	Step	eval_loss	Ghi chú
1	2207	1.3073	
2	4414	1.2203	weights được báo cáo
3	6621	1.2667	bắt đầu overfit

Vì `load_best_model_at_end = True`, bộ trọng số bàn giao ở `models/phobert_qa/` là của epoch 2 chứ không phải epoch cuối; `sha256sum` xác nhận nó khớp với `checkpoint-4414` và khác `checkpoint-6621`. Mọi số liệu trong báo cáo đều được chấm lại từ chính bộ trọng số đã bàn giao.

4 Kỹ thuật đề xuất: cấu trúc một tầng và cấu trúc hai tầng

Hai cấu trúc dùng chung encoder ở mục 3, chỉ khác nhau ở cách quyết định *chọn span* nào và *có nên trả lời hay không*.

4.1 Cấu trúc một tầng

Suy luận theo hậu xử lý chuẩn của SQuAD 2.0 [3], hiện thực trong `src/evaluate.py`, gồm ba bước.

(1) **Liệt kê ứng viên.** Trong mỗi cửa sổ, lấy 20 vị trí z^{start} cao nhất và 20 vị trí z^{end} cao nhất, duyệt tích Cartesian với $i \leq j$ và $j - i + 1 \leq \text{max_answer_length} = 64$. Điểm của một cặp là $\text{score}(i, j) = z_i^{\text{start}} + z_j^{\text{end}}$; cặp tốt nhất trên mọi cửa sổ là `best_score`, và đáp án là `context[start_char : end_char]` khôi phục từ `offset_mapping`.

(2) **Điểm null.** Mỗi cửa sổ cho một điểm “không có đáp án” tại chính token `<s>`:

$\text{null}^{(w)} = z_0^{\text{start}} + z_0^{\text{end}}$. Giá trị quyết định là $\text{null}_{\min} = \min_w \text{null}^{(w)}$, tức “chỉ cần một cửa sổ tin rằng có đáp án thì cứ trả lời”.

(3) Luật quyết định.

$$\text{Dự đoán} = \begin{cases} \emptyset & \text{nếu } \text{null}_{\min} > \text{best_score} \\ \text{span khôi phục từ gốc} & \text{ngược lại} \end{cases}$$

Cả hai đại lượng so với nhau đều là **tổng logit thô**, không phải xác suất, và tầng này **không có ngưỡng nào**. Hành vi “biết nói không biết” vì thế phụ thuộc hoàn toàn vào thang điểm của head pointer: chỉ cần tổng logit của một span dài thừa cao hơn $\text{null}_{\min}$ một chút là hệ thống mở miệng. Đây là chỗ mất nhiều điểm nhất — trên test nó chỉ từ chối đúng 31.78% câu bẫy (mục 5.2).

4.2 Cấu trúc hai tầng

Điểm xuất phát là một số đo: pool 40 ứng viên của extractor có recall 92.82% trên validation (1.796/1.935 câu có đáp án) và 93.40% trên test (1.810/1.938). Ở khoảng 93 trên 100 câu, *đáp án đúng đã nằm sẵn trong danh sách*, chỉ chưa được đưa lên đầu. Vấn đề không phải thiếu thông tin mà là thiếu cách chọn và cách quyết định có nên chọn hay không. Tầng hai đánh vào đúng chỗ đó, trên cùng một encoder và cùng một pool.

(1) **Pool ứng viên.** `src/qa_service.py` gom mọi ứng viên hợp lệ của mọi cửa sổ, sắp giảm dần theo $\text{score}(i, j)$, khử trùng lặp theo cặp tọa độ ký tự (cs, ce) và cắt còn tối đa $\text{top_k_pool} = 40$. Trần 40 *thực sự ràng buộc*: 77.7% câu validation và 75.5% câu test chạm đúng 40 ứng viên (trung bình 37.25 ứng viên/câu, không câu nào dưới 10). Recall trên vì thế một phần do K ; tăng K là đòn chỉnh còn để ngỏ.

(2) **Biểu diễn ứng viên.** Mỗi ứng viên thành một véc tơ 34 chiều do `src/reranker_features.py` sinh ra; sáu nhóm đặc trưng ứng với sáu giả thuyết lỗi ở mục 5.2.

Table 6: Ba mươi tư đặc trưng bề mặt của một ứng viên, theo sáu nhóm; cột **Chiều** là số thành phần đóng góp vào véc tơ 34 chiều

Nhóm	Nhãn	Chiều	Nội dung
Điểm & hạng	(i)	4	$\text{score}(i, j)$, hạng trong pool, độ lệch so với ứng viên đầu bảng, z-score trong pool
Độ dài	(ii)	7	số token, số từ, số ký tự, log tỉ lệ độ dài so với median độ dài đáp án tập train và giá trị tuyệt đối của nó, cờ “vượt p95”, cờ “một token”
Dạng bề mặt	(iii)	7	hư từ tiếng Việt ở đầu/cuối span, dấu câu ở biên, số dấu phẩy, có dấu chấm, hoa chữ cái đầu, chứa số
Khớp câu hỏi	(iv)	2	span có chứa một từ để hỏi, tỉ lệ trùng token với câu hỏi
Hình học pool	(v)	7	log kích thước pool, chỉ số và log số của số, bị ứng viên điểm cao hơn <i>bao chứa</i> (dấu hiệu cắt biên kinh điển), số ứng viên thấp hơn bị nó bao chứa, độ dài khoảng cách tới span bao, có biến thể biên hay không
Quan hệ với null	(vi)	7	ba hiệu số giữa $\text{score}(i, j)$ và $\text{null}_{\min}$, $\text{null}_{\max}$, top_score ; cộng với bốn slot chỉ hàng NULL mang giá trị thật (<code>is_null</code> , $\text{null}_{\min}$, $\text{null}_{\max}$, $\text{top_score} - \text{null}_{\min}$)
Tổng		34	30 chiều dùng chung cho mọi hàng, 4 chiều riêng cho hàng NULL

(3) Ứng viên NULL bắt buộc. Ngoài k ứng viên thật, một hàng giả `\x00NULL` luôn được thêm vào cuối ma trận: nó mang cờ `is_null` cùng ba giá trị $\text{null}_{\min}$, $\text{null}_{\max}$, $\text{top_score} - \text{null}_{\min}$ ở bốn slot dành riêng (bốn slot này bằng 0 trên mọi ứng viên thật) và 0 ở toàn bộ đặc trưng bề mặt. Bài toán vì thế thành *phân loại nhị phân* “hàng này có phải đáp án đúng không” trên $k + 1$ hàng, để phương án “im lặng” thi đua bình đẳng với mọi span.

(4) Mô hình chấm lại. `HistGradientBoostingClassifier` của scikit-learn [11], một cài đặt gradient boosting trên cây hồi quy [9]. Reranker huấn luyện trên pool của **tập train** — 22.702 câu trả lời thành 865.310 hàng ứng viên, trong đó 3.24% mang nhãn 1 — bằng đúng bộ đặc trưng trên. Nhãn của một hàng là 1 nếu span trùng khít (sau chuẩn hoá) với đáp án vàng; hàng NULL nhận nhãn 1 khi và chỉ khi câu hỏi `is_impossible`. Vì dương tính chỉ chiếm khoảng một phần ba mươi, `class_weight = balanced` được bật; cấu hình cây là `max_iter = 300`, `learning_rate = 0.08`, `max_depth = 6`, `early_stopping` với `validation_fraction = 0.1`, `seed = 42`. Cây trên đặc trưng bề mặt được chọn thay vì mô hình thần kinh thứ hai vì nó chạy trên CPU nên không tranh GPU với encoder, và độ quan trọng từng đặc trưng đọc được trực tiếp.

(5) **Luật từ chối có hiệu chuẩn.** Với $\hat{p}$ là xác suất lớp 1 reranker trả về:

$$\text{abstain} \iff (\mathcal{P} = \emptyset) \vee (\hat{p}_{\text{null}} > \hat{p}_{\text{best}} \wedge \hat{p}_{\text{null}} \geq \tau), \quad \tau = 0.95$$

Cần *cả hai* về vì xác suất của mô hình tăng cường độ lệch bão hoà sát 1; nếu chỉ so sánh hơn-kém thì hệ thống “im lặng” quá đà. Ngưỡng τ biến quyền từ chối thành **một nút xoay đơn điệu**: tăng τ luôn đổi câu bấy lấy F_1 câu có đáp án. Nút này quay miễn phí: điểm reranker không đổi theo τ , mỗi câu hỏi encoder chỉ chạy **đúng một lần**. Dải có tác dụng đo ở mục 5.3.

Algorithm 1 Suy luận hai tầng với quyền từ chối

Require: Câu hỏi Q , ngữ cảnh C , $K = 40$, ngưỡng τ

Ensure: Một span của C hoặc “không có đáp án”

- 1: $\mathcal{P} \leftarrow \emptyset$
 - 2: **for all** cửa sổ w của (Q, C) **do**
 - 3: $(\mathbf{z}^{\text{start}}, \mathbf{z}^{\text{end}}) \leftarrow \text{PhoBERT}(w)$
 - 4: Tạo các span hợp lệ từ top-20 start/end và thêm vào $\mathcal{P}$
 - 5: **END FOR EACH**
 - 6: Giữ top- K ứng viên sau khi khử trùng lặp
 - 7: Tạo đặc trưng 34 chiều và thêm hàng NULL
 - 8: $\hat{\mathbf{p}} \leftarrow \text{HISTGB}(X)$
 - 9: **IF** $\mathcal{P} = \emptyset$ hoặc $\hat{p}_{\emptyset} > \max_{c \in \mathcal{P}} \hat{p}_c$ và $\hat{p}_{\emptyset} \geq \tau$ **then**
 - 10: **RETURN** “không có đáp án”
 - 11: **END IF**
 - 12: **RETURN** span có xác suất lớn nhất, khôi phục từ C
-

Phạm vi hiệu lực của tầng hai cần nói rõ. Trên validation có 821 câu mà đáp án vàng *đã* nằm sẵn trong pool nhưng extractor không xếp lên đầu: reranker đặc trưng bề mặt chỉ sửa được đúng 12 câu. Khi head pointer chấm z^{start} và z^{end} độc lập, lỗi biên là lỗi kiến trúc, không chữa được bằng hậu xử lý bề mặt. Giá trị đo được của tầng hai — và là luận điểm chính của đồ án — nằm ở vế thứ hai của thiết kế: **biến một so sánh logic thô thành một xác suất có ngưỡng điều khiển được**.

4.3 Bảng so sánh hai cấu trúc

Table 7: Đối chiếu thiết kế giữa cấu trúc một tầng và cấu trúc hai tầng

Khía cạnh	Một tầng	Hai tầng
Mô-đun	Encoder + head pointer + hậu xử lý <code>src/evaluate.py</code>	Thêm <code>QAService</code> : pool, 34 đặc trưng, HISTGB
Không gian tìm kiếm	Span tốt nhất của mỗi cửa sổ, so với nhau bằng $\text{score}(i, j)$	Pool 40 ứng viên đã khử trùng lặp trên mọi cửa sổ
Bản chất điểm số	Tổng logit thô, không phải xác suất, không so sánh được giữa các câu	Xác suất $[0, 1]$ rằng <i>đúng hàng này</i> là đáp án đạt
Có “hàng NULL” không	Không — null chỉ là một điểm để so sánh, không phải một phương án được chấm	Có — hàng <code>\x00NULL</code> thi đua bình đẳng với các span
Luật im lặng	$\text{null}_{\min} > \text{best_score}$	$\hat{p}_{\text{null}} > \hat{p}_{\text{best}}$ và $\hat{p}_{\text{null}} \geq \tau$
Tham số điều khiển	Không có	Một: τ (mặc định 0.95)
Chi phí GPU	Encoder chạy 1 lần/câu	Không đổi: encoder vẫn chạy 1 lần/câu; reranker chạy CPU
Nhóm lỗi nhầm tới	—	853 câu sai ở <i>quyết định có trả lời</i> (mục 5.2)
Điểm yếu còn lại	Cả hai nhóm lỗi	Nhóm 608 câu cắt sai biên, do head pointer quyết định

5 Kết quả thực nghiệm và thảo luận

5.1 Thiết lập đánh giá

Độ đo theo chuẩn SQuAD, hiện thực trong `src/evaluate.py`. Hàm `normalize_text` gồm ba bước: chuyển chữ thường, **xóa toàn bộ dấu câu**, nén khoảng trắng; không bỏ mạo từ như bản SQuAD tiếng Anh vì tiếng Việt không có mạo từ.

- **EM** = 1 nếu chuỗi đã chuẩn hoá của dự đoán trùng khít đáp án vàng, ngược lại 0.
- **F1 theo token**: với P và G lần lượt là tập token dự đoán và tập token vàng,

$$\text{F1} = \frac{2 \cdot |P \cap G|}{|P| + |G|}$$

cả hai đều rỗng thì $\text{F1} = 1$, một trong hai rỗng thì $\text{F1} = 0$.

- **HasAns EM/F1**: tính riêng trên câu có đáp án. **NoAns Accuracy**: tỉ lệ câu bẫy mà hệ thống bỏ trống. Hai nhóm sau bắt buộc báo cùng nhau, vì hệ thống luôn trả lời sẽ có $\text{NoAns} = 0$ nhưng **HasAns** bị thổi phồng.

Môi trường. Toàn bộ số liệu sinh ra trên một máy cá nhân duy nhất, `seed = 42` cho mọi thí nghiệm.

Table 8: Môi trường thực nghiệm

Thành phần	Phiên bản đã cài khi đo
GPU	NVIDIA GeForce RTX 3050 Laptop, 4096 MiB
Python / PyTorch	3.10.11 / 2.9.1+cu128
Transformers / Datasets	4.57.1 / 4.2.0
scikit-learn / joblib	1.7.2
numpy / pandas	2.2.6 / 2.3.3
Streamlit	1.63.0

5.2 Chất lượng của mô hình nền trên test ($n = 2882$)

Bảng 9 là output của `src/evaluate.py` trên `data/processed/test.parquet`, đã tái kiểm tra bằng cách chấm lại từng câu trong `predictions.json`.

Table 9: Kết quả mô hình một tầng trên UIT-ViQuAD 2.0 test

Độ đo	Giá trị
EM toàn bộ	42.40
F1 toàn bộ	57.90
EM câu có đáp án (HasAns)	47.57
F1 câu có đáp án (HasAns)	70.62
Độ chính xác câu bẫy (NoAns)	31.78
Tỉ lệ hệ thống chịu trả lời	82.34
F1 riêng trên 1729 câu có đáp án mà hệ thống chịu trả lời	79.16

Hàng cuối định hướng toàn bộ phần còn lại: *khi hệ thống chịu nói, nó nói rất đúng* ($F_1 = 79.16$). Khoảng cách xuống 70.62 rồi 57.90 không đến từ việc thiếu kiến thức, mà từ việc không biết dừng lại. Để kiểm chứng bằng định lượng, 2.882 câu được phân hoạch thành tám nhóm loại trừ nhau, so chuỗi đã chuẩn hoá của dự đoán với đáp án vàng.

Table 10: Hồ sơ lỗi của mô hình một tầng trên test ($n = 2882$, các nhóm loại trừ nhau)

Nhóm lỗi	SL	%	Bản chất
Đúng (có đáp án)	922	31.99	EM = 1
Bịa đáp án cho câu bẫy	644	22.35	trả lời một câu không có đáp án
Cắt dài thừa	375	13.01	span chứa đáp án vàng cộng thêm nhiều
Bỏ đúng câu bẫy	300	10.41	từ chối đúng
Cắt ngắn thiếu	233	8.08	span chỉ là một phần đáp án vàng
Từ chối oan	209	7.25	bỏ trống câu thực ra có đáp án
Trùng một phần, sai biên	110	3.82	overlap token nhưng không bao chứa
Sai hoàn toàn	89	3.09	không giao nhau về token
Tổng hai nhóm cắt sai biên	608	21.10	375 + 233

Hồ sơ lỗi tách thành hai bài toán:

- **Quyết định có trả lời hay không:** 644 câu bịa + 209 câu từ chối oan = 853 câu, tức 29.6% test — nơi F_1 bị hụt nhiều nhất so với 79.16 và là nơi luật có τ can thiệp được.
- **Chọn đúng biên của span:** 608 câu (21.10%), nằm ở head pointer chứ không ở luật quyết định. Tầng hai giải quyết được bài toán thứ nhất, không với tới bài toán thứ hai.

5.3 So sánh một tầng và hai tầng bằng số liệu

Hai cấu trúc được chấm trên cùng một *pipeline*, cùng một luật hậu xử lý, cùng một lần *encode*, chỉ khác nhau sự có mặt của reranker, trên mẫu 200 câu test rút ngẫu nhiên ($\text{seed} = 42$, `src/batch_eval.py`, $\tau = 0.95$).

Table 11: Một tầng (pointer thuần) và hai tầng (reranker + ngưỡng $\tau = 0.95$) — test, $n = 200$

Độ đo	Một tầng	Hai tầng	Δ
EM toàn bộ	41.50	46.00	+4.50
F1 toàn bộ	56.69	58.92	+2.23
EM câu có đáp án	44.78	41.04	-3.73
F1 câu có đáp án	67.45	60.33	-7.12
Độ chính xác câu bẫy	34.85	56.06	+21.21
Tỉ lệ bỏ trả lời	22.00	35.50	+13.50

Cột “một tầng” (41.50 / 56.69) sát số toàn test (42.40 / 57.90), nên mẫu 200 câu không lệch và phép so sánh công bằng.

Điều không đạt: reranker **không** làm hệ thống chọn span giỏi hơn — độ đo trực tiếp cho việc chọn span là HasAns F1 *giảm* 7.12 điểm. *Điều đạt,* và là thứ tạo ra +4.50 EM cùng +2.23 F1: **quyền từ chối**, với câu bẫy bắt đúng tăng +21.21 (từ 34.85 lên 56.06) và tỉ lệ bỏ trả lời tăng 22.00% → 35.50%. Cơ chế tăng điểm nằm ở luật quyết định, đúng như hồ sơ lỗi 853 câu đã báo trước.

Vì τ chỉ xuất hiện ở luật quyết định, dải ngưỡng quét được mà không cần chạy lại GPU. Trên validation ($n = 2872$):

Table 12: Quét ngưỡng τ trên validation ($n = 2872$)

τ	EM	F1	Câu bẫy	F1 có đáp án
0.90	46.41	57.89	58.27	57.70
0.95 (mặc định)	45.33	58.76	50.59	62.72
0.97	43.04	58.07	36.93	68.31
0.99	40.46	56.33	24.33	71.83
Không bao giờ từ chối	33.74	50.52	0.00	74.98

Ba nhận xét từ bảng:

- **Đỉnh F_1 nằm tại $\tau = 0.95$,** nên giá trị này thành mặc định trong DEFAULT_TAU_NULL.
- **Đường cong đơn điệu và đổi chỗ được:** tăng τ luôn lấy câu bẫy đổi lấy F_1 có đáp án. Hệ thống “không bao giờ từ chối” mất 8.24 điểm F_1 toàn bộ (50.52 so với 58.76) dù có F_1 câu có đáp án cao nhất bảng. Quyền từ chối vì thế là một phần của hệ thống, không phải một tùy chọn.
- **Núm xoay chỉ có tác dụng ở phần trên của dải:** trên mẫu 200 câu test, chín ngưỡng liên tiếp từ 0.50 đến 0.90 cho kết quả *giống hệt nhau* (47.00 EM, 63.64 câu bẫy, 42.0% bỏ trả lời). Ở $\tau = 1.00$ quyền từ chối biến mất và hai tầng rơi xuống 32.50 EM, *thấp hơn cả một tầng* (41.50 trên cùng 200 câu).

Nói cách khác: toàn bộ điểm của hai tầng đến từ quyền từ chối chứ không từ xếp hạng, và xác suất của reranker chưa hiệu chuẩn tốt.

Độ trễ đo bằng `time.perf_counter` quanh một lần gọi `QAService.answer()` (đã gồm encoder và reranker) trên RTX 3050, FP16: nạp mô hình một lần 17–21 giây; câu ngắn (một cửa sổ) 40–90 ms; câu thuộc test set 190–915 ms tùy độ dài đoạn, 60 câu hết 16–22 giây.

5.4 Giới hạn

- Phép so sánh hai cấu trúc thực hiện trên mẫu 200 câu, không phải toàn bộ 2.882 câu, do giới hạn thời gian trên GPU 4GB. Mẫu khớp với toàn test ở cột nền, nhưng các giá trị Δ vẫn mang sai số lấy mẫu.

-
- Reranker học từ pool tập train, τ chọn trên validation; **test set không tham gia vào bất kỳ bước nào của tầng hai**. Validation vẫn “chạm” vào hệ thống ở hai chỗ — chọn τ và chọn trọng số encoder theo `eval_loss` — nên điểm test không hoàn toàn miễn phí với validation, dù không phải điểm tự chấm trên dữ liệu huấn luyện.
 - AUC 0.929 của reranker trên tập do chính nó huấn luyện là con số lạc quan quá mức; nó bị loại khỏi báo cáo ngay khi được xác định là hiện tượng thẩm định vòng lặp.
 - Không có đường cơ sở ngoài: mọi so sánh đều nội bộ giữa các biến thể của cùng một hệ thống. F_1 test 57.90 nên hiểu là kết quả *tái lập được* trong môi trường mục 5.1, không phải một thứ hạng trên bảng công bố.
 - Chia tách bảo đảm 0 ngữ cảnh chung giữa train và test nhưng không rời khỏi miền chủ đề (110/138 `title` chung hai tập), nên điểm test phản ánh năng lực đọc hiểu trong miền đã thấy chủ đề, không phải năng lực tổng quát hoá liên miền.

6 Ứng dụng web

6.1 Kiến trúc: một lõi suy luận, hai giao diện

Toàn bộ logic suy luận nằm trong `QAService` (`src/qa_service.py`, 249 dòng) và **chỉ nằm ở đó**: encoder, cửa sổ trượt, pool 40 ứng viên và reranker đều đi qua đúng một hàm `answer(question, context, top_k, tau_null)`. Hai lớp vỏ chỉ khác cách bày: `app/app.py` (468 dòng) là giao diện Streamlit [12] cho người dùng cuối, `src/api.py` (109 dòng) là REST API FastAPI cho chương trình. Nhờ chung một lõi, số liệu trên web và số liệu `src/batch_eval.py` chấm hàng loạt là *cùng một đường chạy*. Mô hình nạp một lần bằng `@st.cache_resource` và được mọi thao tác sau dùng lại.

Giao diện có hai tab.

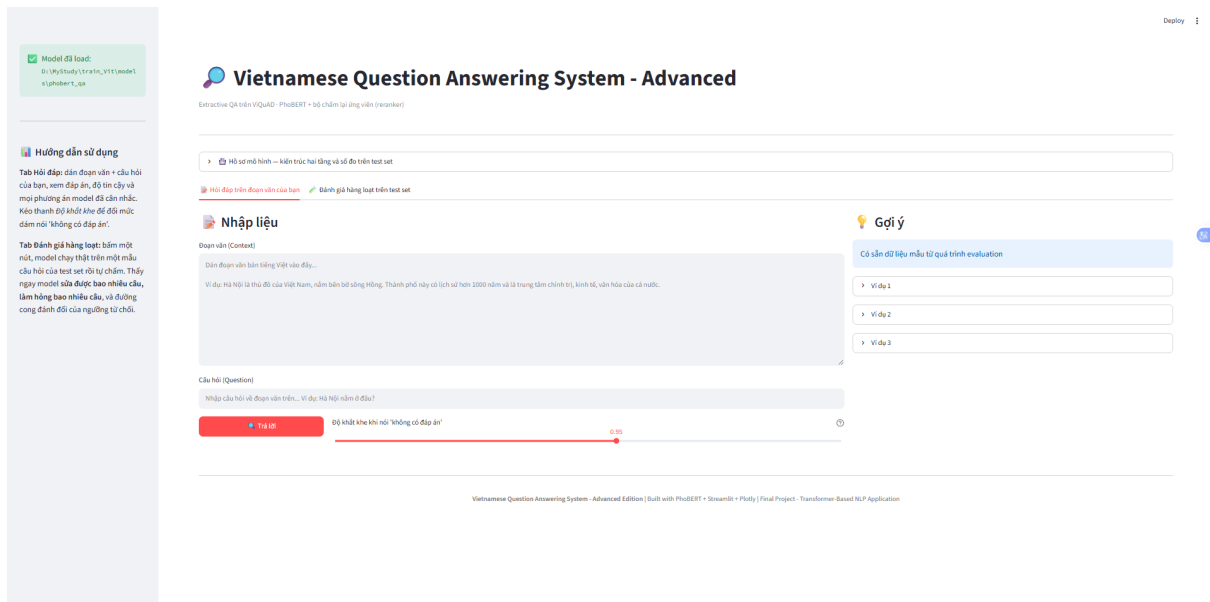


Figure 1: Tab 1 — Hỏi đáp trên đoạn văn của người dùng

Tab 1 — Hỏi đáp: Người dùng nhập ngữ cảnh và câu hỏi, điều chỉnh τ . Hệ thống trả về đáp án hoặc từ chối; kết quả gồm độ tin cậy, số ứng viên, số cửa sổ và độ trễ. Đáp án được tô đậm trong ngữ cảnh gốc.

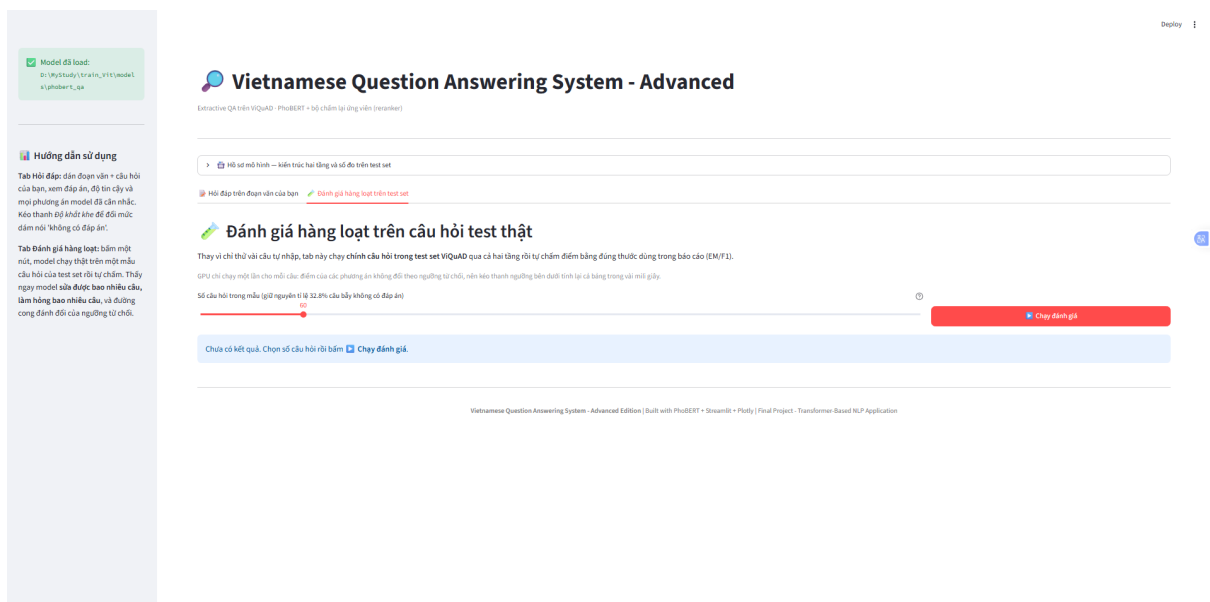


Figure 2: Tab 2 — Đánh giá hàng loạt trên test set thật

Tab 2 — Đánh giá hàng loạt: Chạy các câu hỏi trên test set qua hai cấu trúc và tính các độ đo. Người dùng có thể thay đổi τ , xem so sánh từng câu và xuất kết quả CSV.

6.2 Triển khai và giới hạn của bản demo

Table 13: Hai endpoint của `src/api.py` (đã kiểm chứng bằng `curl`)

Method	Đường dẫn	Nội dung
GET	<code>/health</code>	Trạng thái, thiết bị suy luận, số tham số encoder.
POST	<code>/answer</code>	Vào: <code>question</code> , <code>context</code> , <code>top_k</code> , <code>tau_null</code> . Ra: <code>answer</code> , <code>abstained</code> , <code>confidence</code> , danh sách ứng viên kèm <code>proba</code> và tọa độ ký tự, <code>null_proba</code> , số cửa sổ, và phương án mô hình một tầng sẽ chọn.

Giới hạn của bản demo: ứng dụng đơn tiến trình, không xác thực, không hàng đợi, nên tải đồng thời sẽ xếp hàng trước cùng một GPU; trọng số 537 MB nằm ngoài mã nguồn, người chạy phải tự đặt `models/phobert_qa/` và `models/reranker/reranker.pkl` đúng đường dẫn.

7 Kết luận

Đồ án đã xây được một hệ thống hỏi–đáp trích xuất tiếng Việt chạy từ đầu đến cuối trên một GPU 4GB, và do được hệ thống đó mạnh, yếu ở đâu.

Pipeline ánh xạ ký tự–token và cửa sổ trượt được tự viết và tái lập hoàn toàn, kèm bước sửa 153 vị trí đáp án lệch trong dữ liệu gốc. *Chất lượng nền* trên UIT-ViQuAD 2.0 test: $EM = 42.40$, $F_1 = 57.90$, HasAns $F_1 = 70.62$, NoAns = 31.78, và $F_1 = 79.16$ trên 1.729 câu có đáp án mà hệ thống chịu trả lời. *Hồ sơ lỗi* tám nhóm phân hoạch đủ 2.882 câu cho thấy hai khiếm khuyết lớn nhất là *không dám im lặng* (853 câu) và *cắt sai biên* (608 câu), không phải thiếu năng lực định vị thông tin. *Kỹ thuật đề xuất* — cấu trúc hai tầng với ngưỡng từ chối hiệu chuẩn τ — xử lý đúng khiếm khuyết thứ nhất: +21.21 điểm bắt câu bẫy và +4.50 EM trên mẫu so sánh, đổi lấy -7.12 HasAns F_1 ; toàn bộ đánh đổi điều khiển được bằng một tham số duy nhất ngay trên giao diện.

Hướng phát triển tiếp theo nằm ở kiến trúc:

1. Thay head pointer bằng **span scorer** chấm trực tiếp từng cặp (i, j) thay vì cộng rời hai logit. Cơ sở là việc 92.82% đáp án đúng đã nằm sẵn trong pool nhưng hậu xử lý bề mặt chỉ sửa được 12/821 câu; nếu thử nghiệm nhỏ cho thấy span scorer không thắng pointer thì nên bỏ.
2. Reranker bằng **biểu diễn học được** (véc tơ câu hỏi + véc tơ span từ chính encoder) thay cho 34 đặc trưng bề mặt, chỉ đáng làm nếu hướng (1) đúng.
3. **Đo hai tầng trên toàn bộ test** để đóng khoảng sai số lấy mẫu ở mục 5.4.

-
4. **Phát hành trọng số** 537 MB trên nguồn riêng (Google Drive hoặc Hugging Face Hub) vì dung lượng này không phù hợp với kho mã.

8 References

- [1] K. V. Nguyen, D.-V. Nguyen, A. G.-T. Nguyen và N. L.-T. Nguyen, “UIT-ViQuAD: A manually reviewed Vietnamese dataset for extractive question answering”, *Proceedings of the 28th International Conference on Computational Linguistics (COLING 2020)*, tr. 2595–2605, 2020.
- [2] K. V. Nguyen, S. Q. Tran, L. T. Nguyen, T. V. Huynh, S. T. Luu và N. L.-T. Nguyen, “VLSP 2021 – ViMRC Challenge: Vietnamese Machine Reading Comprehension”, *The 8th International Workshop on Vietnamese Language and Speech Processing (VLSP 2021)*, arXiv:2203.11400, 2021.
- [3] P. Rajpurkar, R. Jia và P. Liang, “Know What You Don’t Know: Unanswerable Questions for SQuAD”, *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018)*, tr. 1071–1076, 2018.
- [4] D. Q. Nguyen và A. T. Nguyen, “PhoBERT: Pre-trained language models for Vietnamese”, *Findings of the Association for Computational Linguistics: EMNLP 2020*, tr. 1012–1025, 2020.
- [5] Y. Liu và cộng sự, “RoBERTa: A Robustly Optimized BERT Pretraining Approach”, arXiv:1907.11692, 2019.
- [6] J. Devlin và cộng sự, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”, *Proceedings of NAACL-HLT 2019*, tr. 4171–4186, 2019.
- [7] I. Loshchilov và F. Hutter, “Decoupled Weight Decay Regularization”, *International Conference on Learning Representations (ICLR 2019)*, arXiv:1711.05101, 2019.
- [8] T. Chen, B. Xu, C. Zhang và C. Guestrin, “Training Deep Nets with Sublinear Memory Cost”, arXiv:1604.06174, 2016.
- [9] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine”, *The Annals of Statistics*, tập 29, số 5, tr. 1189–1232, 2001.
- [10] T. Wolf và cộng sự, “Transformers: State-of-the-Art Natural Language Processing”, *Proceedings of EMNLP 2020 – System Demonstrations*, tr. 38–45, 2020.
- [11] F. Pedregosa và cộng sự, “Scikit-learn: Machine Learning in Python”, *Journal of Machine Learning Research*, tập 12, tr. 2825–2830, 2011.
- [12] Streamlit Inc., “Streamlit Documents – caching guide”, <https://docs.streamlit.io/>, truy cập 2026.